A Brief Introduction on Support Vector Machines

James Skripchuk

December 15, 2018

1 Introduction

Machine Learning is a branch of Computer Science that aims to construct algorithms and models that can be iteratively trained to optimize the results of a certain task or to learn features about a data set. One such task is the classification of data, which occurs in a multitude of disciplines. For example - given a set of health data for an individual, are they at risk at developing cancer? In this essay, I will give a brief introduction to a specific machine learning model called a *support vector machine*, and discuss the concept behind the model and the mathematics behind it.

A support vector machine (SVM) is a *supervised binary classifier*. A supervised machine learning model is one where the data used to train and construct the model (*training data*) is classified beforehand, and the model uses this data to infer how to classify a new unknown instance. Binary classifiers are a specific type of classifier that, when given a new data instance to classify, will place it in one of two distinct classes. An example supervised binary classifier would be a model that was trained on a set of patient health data from a hospital, where each patient has many numerical health factors (such as pulse, blood pressure, weight, etc.) and is labeled as healthy or not healthy. Given a new patient, the model will use the numerical data of the patient and then classify the patient as healthy or not healthy.

Let us define the concept of a binary classifier more formally. A binary classifier is a function $f(x)$, that when given a vector x with numerical attributes $[x_1, x_2, \ldots x_n]$, $f(x)$ will return result y where $y \in \{-1, 1\}$. 1 will denote a positive result, while -1 will denote a negative result.

The way a support vector machine goes about this classification is as follows - given a set of n-dimensional data, the support vector machine will attempt to linearly separate the data using a single $(n-1)$-dimensional hyperplane.[1] When classifying a new instance, if it lands on the "positive" side of the hyperplane it is classified as positive, and vice versa for the negative side. In all our final function $f(x)$

$$f(x) = g(w^T x + b) \tag{1}$$

will be dependent on the normal vector w of our separating hyperplane and an offset vector b. To bind our output to $\{-1, 1\}$, $g(z) = 1$ if $z \geq 0$ and $g(z) = -1$ if $z < 0$.

[1] We will discuss the case of non-linearly separable data later

However, there are theoretically infinite ways we could define this separating hyperplane (as shown in Figure 1), so how do we choose the best one? In Figure 1, many of these hyperplanes seem to be unnecessarily close to our training data. If we were classifying an instance that was right next to a cluster of training data, but just over the hyperplane, it would be classified as the other class. Intuitively, we want a wide gap between the hyperplane and our two classes so that clusters of data will remain distinct.

For our separating hyperplane, we want the largest amount of space between the hyperplane itself and our training data. The distance between a training instance and the hyperplane is called the *margin*, so thus what we want is called a *maximal margin classifier*. Essentially, we want to find w and b such that the minimum of all the margins in the training set is maximized. Finding the hyperplane that will give us maximum margin is a question of mathematical optimization, so our plan of attack will be to transform our support vector machine problem into a set of constraints and equations that can be solved by commercially available optimization software.

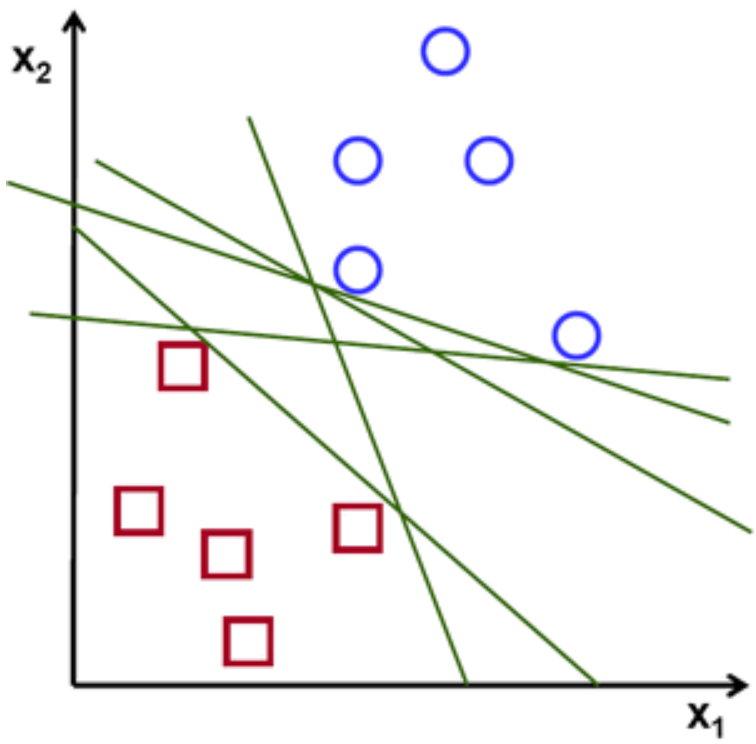

Figure 1: A SVM in 2D space. There are many possible separating hyperplanes. [2]

2 The Model

There are two types of margins that are discussed in support vector machines: *functional margins* and *geometric margins*. The functional margin is hard to gain an intuition for, but it is needed later in the derivation so we will define it now. If we have a set of m training instances, the i-th training instance will be represented as the 2-tuple $(x^{(i)}, y^{(i)})$ - where $x^{(i)}$

is a vector of attributes of the instance and $y^{(i)} \in \{-1, 1\}$ is the label of the instance given beforehand. The functional margin of the i-th training instance is

$$\hat{\gamma}^{(i)} = y^{(i)}(w^T x + b) \tag{2}$$

If our prediction is correct and confident, the functional margin will be large and greater than zero. This is due to the fact that if $y^{(i)} = 1$, the instance will be above the boundary, thus $w^T x + b > 0$. Conversely, if $y^{(i)} = -1$, the instance is below the boundary and $w^T x + b < 0$. We will talk more about the functional margin later, but we will now turn our attention to the more visual geometric margin.

The geometric margin is a scalar $\gamma^{(i)}$ that represents the distance from a training instance $x^{(i)}$ (represented by point A on Figure 2) from our hyperplane. The decision boundary from the equation $w^T x + b = 0$ is also shown. Our end goal is that we want to maximize the minimum of all of the geometric margins, so in order to do this we first must find a general method to solve for any geometric margin $\gamma^{(i)}$ given a training instance $x^{(i)}$ relative to our decision boundary.

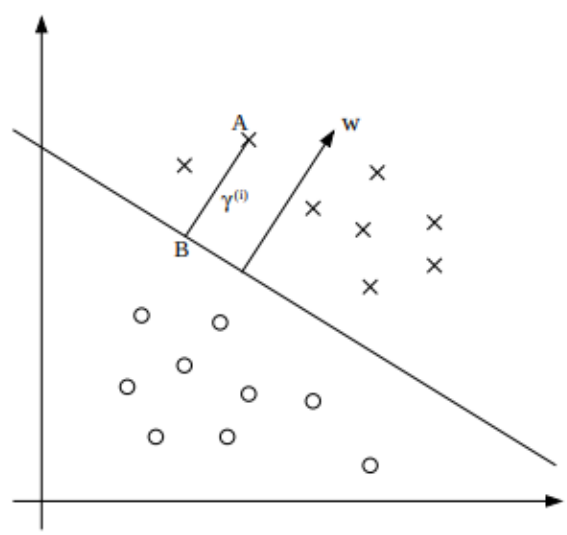

Figure 2: Solving for $\gamma^{(i)}$[1]

We know that $w/||w||$ is a unit vector that points in the same direction as w, so thus point B (Figure 2) on the decision boundary can be represented as $B = x^{(i)} - \gamma^{(i)}(w/||w||)$. Since B is on the decision boundary, we know that $w^T B + b = 0$. Substituting B in gives us

$$w^T \left(x^{(i)} - \gamma^{(i)} \frac{w}{||w||} \right) + b = 0 \tag{3}$$

and then solving for $\gamma^{(i)}$ gives us

$$\gamma^{(i)} = \left(\frac{w}{||w||} \right)^T x^{(i)} + \frac{b}{||w||} \tag{4}$$

We want our geometric margin to be positive if our instance is on the correct side of the decision boundary, so we just multiply by $y^{(i)}$.

$$\gamma^{(i)} = y^{(i)} \left(\left(\frac{w}{||w||} \right)^T x^{(i)} + \frac{b}{||w||} \right) \tag{5}$$

If $||w|| = 1$, the geometric margin is equal to the functional margin. So thus given any instance $(x^{(i)}, y^{(i)})$, we can find its geometric margin.

Finally, not losing sight of our goal, we define γ to be the minimum of all the geometric margins in our training set. So thus, given a training set of size m containing instances $(x^{(i)}, y^{(i)})$

$$\gamma = \min_{i=1\ldots m} \gamma^{(i)} \tag{6}$$

So thus, we can now pose the basic form of our optimization problem. We want to find w and b such that the hyperplane defined by $w^T x + b$ maximizes the minimum geometric margin γ for our training set.

3 Optimization and Lagrange Duality

Unfortunately, we are not quite done with our optimization problem. In its current form it is defined as:

$$\max_{\gamma,w,b} \quad \gamma \tag{7a}$$

$$\text{subject to} \quad y^{(i)}(w^T x^{(i)} + b) \geq \gamma, i = 1 \ldots m \tag{7b}$$

$$||w|| = 1. \tag{7c}$$

We want to find the w and b that will give us largest γ. We also constrain $||w|| = 1$ to keep the geometric margin equal to our functional margin. However, due to the nature of optimization algorithms, in this form our problem is not easy to solve. Most optimization software can only reliably solve convex functions, and the constraint $||w|| = 1$ is not convex. Transforming the problem a bit, we use the fact that $\gamma = \hat{\gamma}/||w||$

$$\max_{\hat{\gamma},w,b} \quad \frac{\hat{\gamma}}{||w||} \tag{8a}$$

$$\text{subject to} \quad y^{(i)}(w^T x^{(i)} + b) \geq \hat{\gamma}, i = 1 \ldots m \tag{8b}$$

$$\tag{8c}$$

Still, $\hat{\gamma}/||w||$ is non convex. However, we can use the fact that scaling w and b by the same constant does not change the functional margin. So we now add the constraint that

$\hat{\gamma} = 1$. We can use that and the fact that maximizing $\hat{\gamma}/||w|| = 1/||w||$ is the same thing as minimizing $||w||^2$, we get the final form of

$$\min_{w,b} \quad \frac{1}{2}||w||^2 \tag{9a}$$

$$\text{subject to} \quad y^{(i)}(w^T x^{(i)} + b) \geq 1, i = 1 \ldots m \tag{9b}$$

$$\tag{9c}$$

This form of the problem can now be solved easily by commercial mathematical optimization software. The exact method of optimization is outside the scope of the paper and more detail can be found in [1], but here we will discuss the important parts.

The optimization problem can be solved using the method of Lagrange multipliers. We construct the Lagrangian to be

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}||w||^2 - \sum_{i=1}^{m} \alpha_i [y^{(i)}(w^T x^{(i)} + b) - 1] \tag{10}$$

Through copious amounts of partial derivatives, we eventually discover that

$$w^T x + b = \sum_{i=1}^{m} \alpha_i y^{(i)} \langle x^{(i)}, x \rangle + b \tag{11}$$

The above equation is much more important than one would think on first glance. The above equation means that in order to classify a new instance x, we only need to compute the dot products of the new instance x with all of the training instances in the training set. Moreover, using another derivation outside the scope of the paper using *KKT Conditions*, we also discover that many of the α's will be equal to 0 except for a few training instances called *support vectors*.

Thus, to classify an instance, we only need to compute the dot products of the instance we want to classify with the support vectors, not the entire training set. This drastically reduces the amount of computation that the support vector machine algorithm will have to do, and thus make it very efficient to learn in high dimensional spaces. These two facts will allow us take advantage of something called *the kernel trick*, which allow us to efficiently compute in high dimensional spaces.

4 The Kernel Trick

We will now address the situation where our data is not linearly separable, and this is where the beauty of the support vector machines shines. If our data is not linearly separable in its input space, we can transform the data into a higher dimensional *feature space* where it is linearly separable, find the hyperplane in the feature space, and then transform back to our original input space. This is best illustrated visually.

We will denote the function that transforms our input vector into a higher dimensional feature space $\phi(x)$. In Figure 3, $\phi(x) : \mathbb{R}^2 \mapsto \mathbb{R}^3$. More specifically, $\phi(x) : (x_1, x_2) \mapsto$

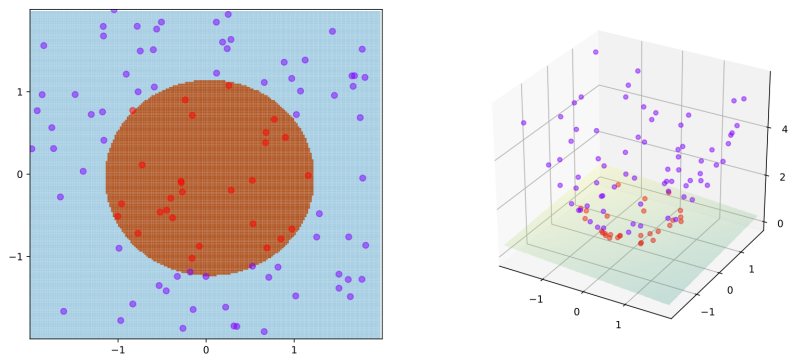

Figure 3: Mapping from a 2-dimensional input space to a 3-dimensional feature space. [3]

$(x_1, x_2, x_1^2 + x_2^2)$. In $\mathbb{R}^3$, it is easy to see that the data can be split by a single plane. When the equation of that plane is graphed back in $\mathbb{R}^2$, it becomes the red ellipse. In general, we can define a function called the *kernel function* that operates on two instance attribute vectors x and z as

$$K(x, z) = \phi(x)^T \phi(z) \tag{12}$$

Now, since we discovered above that our SVM algorithm can operate using only the inner product $\langle x, z \rangle$, we can learn in feature space by just calculating $\langle \phi(x), \phi(z) \rangle$. There are many different types of kernel functions that are used in practice, such as polynomial transformations or transformations based on the "distance" between two data points such as the Gaussian kernel G.

$$G(x, z) = \exp\left(-\frac{||x - z||^2}{2\sigma^2}\right) \tag{13}$$

But still, even if we're only computing dot products, what if our feature space is very high dimensional (in the order of thousands)? To find and compute the transformation $\phi(x)$ for our data points will be still be very expensive. Is there a way we can compute in a higher-dimensional space without finding and computing $\phi(x)$?

This is where things start getting interesting. With support vector machines, there are some high dimensional kernels that give the same result when calculated in a lower dimensional input space as the high dimensional feature space. This is best illustrated with an example (adopted from C. Ranjan's example on Quora [4]).

Let's say $\phi(x) : (x_1, x_2) \mapsto (x_1^2, x_2^2, \sqrt{2}x_1x_2)$. Thus,

$$\langle \phi(x), \phi(z) \rangle \tag{14a}$$

$$\langle (x_1^2, x_2^2, \sqrt{2}x_1x_2), (z_1^2, z_2^2, \sqrt{2}z_1z_2) \rangle \tag{14b}$$

$$x_1^2 z_1^2 + x_2^2 z_2^2 + 2x_1x_2z_1z_2 \tag{14c}$$

However, what if we compute $\langle x, z\rangle^2$?

$$\langle x, z\rangle^2 \tag{15a}$$
$$\langle (x_1, x_2), (z_1, z_2)\rangle^2 \tag{15b}$$
$$(x_1 z_1 + x_2 z_2)^2 \tag{15c}$$
$$x_1^2 z_1^2 + x_2^2 z_2^2 + 2x_1 x_2 z_1 z_2 \tag{15d}$$

We get the *same result* both ways. In the explicit feature mapping, we had to perform 6 operations to transform our data into the feature space, and 3 more in order to compute the dot product, for a total of 9 operations. In the second derivation, we perform 2 operations for the dot product, and 1 more for squaring our result. This gives a total of 3 operations.

Described above is what's called the *kernel trick*, and it is the core of most support vector machine implementations. This was just a small example, but imagine if the polynomial transformation was raised to a very large power (such as hundreds or thousands). By using certain kernel functions, we can bypass mapping our data into higher dimensions and just compute on our low dimensional data directly, saving us orders of magnitudes of operations and giving us the power to train in extremely high dimensional spaces.

5 Conclusion

Support vector machines are a type of machine learning model that perform binary classification. By going through the derivation, we discovered that support vector machines can compute in high dimensional spaces very easily by only needing the inner product of support vectors. More importantly, we learned that some higher dimensional transformations can be calculated using only the dot products in lower dimensional space, thus making the support vector machine a prime choice for classification in high dimensional spaces.

References

[1] A. Ng, *CS229 Lecture Notes: Part V: Support Vector Machines*, `http://cs229.stanford.edu/notes/cs229-notes3.pdf`

[2] OpenCV, `https://docs.opencv.org/2.4/doc/tutorials/ml/ introduction_to_svm/ introduction_to_svm.html`

[3] Wikipedia, `https://en.wikipedia.org/wiki/Kernel_method/media/ File:Kernel_trick_idea.svg`

[4] C. Ranjan, `https://www.quora.com/What-is-the-kernel-trick`